

Programación de Sistemas

Unidad 2: Análisis léxico

Nota de clase 03: Fundamentos del análisis léxico

Manuel Soto Romero

Araceli Liliana Reyes Cabello

Semestre 2026-2

Colegio de Ciencia y Tecnología UACM

En la nota anterior observamos que un compilador no trabaja todo el tiempo con el texto original. Cada fase recibe una representación, realiza una tarea y comunica un resultado. El análisis léxico inicia ese recorrido: lee los caracteres del programa fuente, reconoce unidades elementales y entrega una secuencia de tokens al analizador sintáctico.

En esta nota estudiaremos la función del analizador léxico y distinguiremos tres conceptos relacionados: token, lexema y patrón. También veremos por qué algunos tokens necesitan atributos y cómo se establece la comunicación con el analizador sintáctico. Todavía no construiremos expresiones regulares, autómatas ni un analizador con Lark; esos desarrollos corresponden a otras notas.

Caso inicial. De caracteres a unidades

Retomemos el fragmento ilustrativo utilizado en la nota anterior:

```
valor = valor + inc;
```

Para una persona, el fragmento contiene una asignación y una suma. El analizador léxico comienza con algo más básico: una secuencia de caracteres. Su trabajo consiste en reconocer dónde empieza y termina cada unidad y clasificarla. Una posible salida descriptiva es:

```
ID(valor) ASIG ID(valor) SUMA ID(inc) PYC
```

El fragmento se usa para observar la transformación y **no establece la sintaxis oficial de IMP**. Los nombres de tokens también son etiquetas ilustrativas, no una especificación del lenguaje del curso.

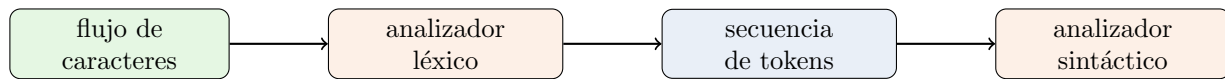
1. Función del analizador léxico en el proceso de traducción

El programa fuente llega al compilador como un flujo de caracteres. Si el analizador sintáctico tuviera que trabajar directamente con cada letra, dígito, espacio y signo, además de reconocer construcciones mayores, su tarea sería más compleja. La separación en fases permite que el analizador léxico se concentre en las unidades elementales y que el analizador sintáctico reciba una entrada más adecuada para reconocer la estructura del programa.

Definición 1. (Analizador léxico)

El **analizador léxico** es el componente del compilador que lee los caracteres del programa fuente, los agrupa en lexemas y produce una secuencia de tokens para el analizador sintáctico.

Su transformación principal puede representarse así:



Además de reconocer lexemas, el analizador léxico suele realizar tareas relacionadas con la lectura del texto:

- ★ omitir espacios, tabuladores, saltos de línea o comentarios cuando las reglas del lenguaje indican que no deben llegar al analizador sintáctico;
- ★ conservar información necesaria sobre los lexemas reconocidos;
- ★ apoyar la ubicación de los elementos del programa para que los diagnósticos puedan relacionarse con el texto fuente.

Estas tareas dependen del lenguaje. Por ejemplo, no siempre es correcto eliminar todos los espacios o saltos de línea: en algunos lenguajes forman parte de la estructura. El analizador léxico debe aplicar la especificación del lenguaje que procesa, no una regla universal.

Separar el análisis léxico del sintáctico ofrece tres ventajas señaladas en las fuentes: simplifica el diseño, permite usar técnicas especializadas para leer y clasificar caracteres, y concentra en un componente los detalles relacionados con la entrada. La separación también establece un límite de responsabilidad.

Corresponde al analizador léxico	Corresponde a fases posteriores
Reconocer unidades a partir de caracteres.	Organizar tokens según la gramática.
Clasificar un lexema en una categoría.	Determinar la estructura sintáctica completa.
Proporcionar los atributos léxicos necesarios.	Comprobar declaraciones, tipos y otras restricciones semánticas.

Observación 1. Reconocer no es interpretar todo el programa

Cuando el analizador léxico encuentra el texto **valor**, puede reconocerlo como un identificador. En ese momento no necesita decidir qué representa el nombre, qué tipo tiene ni si fue declarado correctamente. Esas preguntas requieren información y reglas de otras fases.

2. Del flujo de caracteres a la secuencia de tokens

El analizador léxico avanza sobre la entrada y agrupa caracteres que forman una unidad. El resultado ya no presenta cada carácter por separado, sino una secuencia de elementos clasificados. Para seguir el cambio paso a paso, observemos nuevamente el fragmento inicial.

Caracteres agrupados	Clasificación ilustrativa	Resultado descriptivo
valor	Identificador	ID(valor)
=	Asignación	ASIG
valor	Identificador	ID(valor)
+	Suma	SUMA
inc	Identificador	ID(inc)
;	Terminador	PYC

Los espacios permiten ver con facilidad algunas separaciones, pero no se convierten necesariamente en tokens. En este ejemplo se omiten porque asumimos, sólo para ilustrar el proceso, que no son significativos. Tampoco basta buscar espacios para encontrar todas las fronteras: =, + y ; aparecen junto a otros caracteres y deben reconocerse de acuerdo con las reglas del lenguaje.

El proceso conserva dos clases de información:

- ★ una **clasificación**, que permite distinguir un identificador de un operador o un terminador;
- ★ información sobre la **aparición concreta**, como el texto del identificador o el valor representado por una constante.

La distinción es necesaria porque **valor** e **inc** pertenecen a la misma clasificación, pero son apariciones distintas. Si la salida dijera únicamente ID, las fases posteriores sabrían que encontraron un identificador, pero no cuál.

Ejemplo conceptual. Misma clasificación, distinta información

Consideremos los textos 2 y 25. Ambos pueden clasificarse como números enteros. Sin embargo, no representan el mismo valor. El analizador léxico necesita comunicar la categoría y la información que distingue una aparición de la otra:

$\langle \text{ENTERO}, 2 \rangle$ $\langle \text{ENTERO}, 25 \rangle$

La forma exacta de almacenar esa información es una decisión de diseño. Aquí usamos pares únicamente para hacer visible qué datos se comunican.

3. Tokens, lexemas y patrones

Para describir con precisión el resultado del análisis léxico debemos distinguir tres términos. Están relacionados, pero no son intercambiables.

Definición 2. (Token)

Un **token** es un par formado por un nombre de token y un valor de atributo opcional. El nombre representa una categoría léxica que el analizador sintáctico puede procesar; el atributo conserva información adicional sobre la aparición reconocida.

En exposiciones informales también se usa la palabra *token* para referirse sólo a su nombre o categoría. En esta nota escribiremos el par completo cuando el atributo sea importante y únicamente el nombre cuando no se necesite información adicional.

Definición 3. (Lexema)

Un **lexema** es la secuencia concreta de caracteres del programa fuente que el analizador léxico reconoce como una aparición de un token.

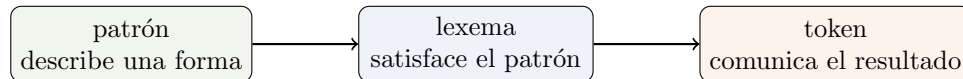
En el ejemplo, las dos apariciones de **valor** son lexemas. Ambas corresponden al nombre de token ID. El texto **inc** es otro lexema que corresponde al mismo nombre de token.

Definición 4. (Patrón)

Un **patrón** es una descripción de la forma que pueden tener los lexemas asociados con un nombre de token.

Por ejemplo, un patrón descrito informalmente como “una letra seguida opcionalmente de letras o dígitos” puede servir para una categoría de identificadores. **valor** e **inc** satisfacen esa descripción; cada uno es un lexema concreto. La escritura formal de los patrones mediante expresiones regulares se estudiará en otra nota.

La relación entre los tres conceptos puede resumirse así:



La tabla siguiente aplica esta relación a ejemplos habituales. Los patrones se expresan en lenguaje natural para no adelantar su notación formal.

Nombre	Patrón informal	Lexema	Token ilustrativo
ID	Letra seguida opcionalmente de letras o dígitos.	valor	⟨ID, valor⟩
ENTERO	Secuencia de uno o más dígitos.	25	⟨ENTERO, 25⟩
SUMA	El carácter +.	+	⟨SUMA⟩
PYC	El carácter ;.	;	⟨PYC⟩

Los operadores y signos de puntuación pueden no requerir atributo porque su nombre ya comunica la información necesaria. En cambio, muchas apariciones pueden satisfacer el patrón de ID o ENTERO; por ello, el atributo permite distinguirlas.

Observación 2. Tres preguntas diferentes

Ante el texto 25, podemos formular tres preguntas:

- ★ ¿Qué caracteres aparecieron? El **lexema** es 25.
- ★ ¿Qué forma satisface? El **patrón** describe una secuencia de uno o más dígitos.
- ★ ¿Qué se comunica? Un **token** posible es ⟨ENTERO, 25⟩.

Responder una pregunta no sustituye las otras dos.

4. Atributos de los tokens y comunicación con el analizador sintáctico

El nombre de un token permite al analizador sintáctico tomar decisiones sobre la estructura. El atributo transporta información particular que puede utilizarse durante la traducción. Por ejemplo, el nombre ENTERO indica la clase de unidad reconocida, mientras que su atributo distingue el valor 2 del valor 25.

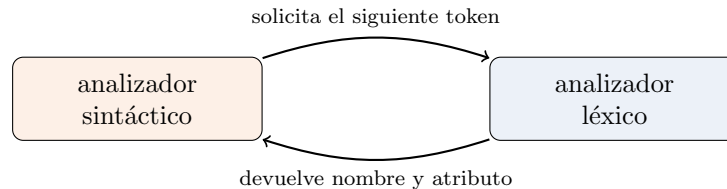
Entre los atributos habituales se encuentran:

- ★ el texto de un identificador;
- ★ el valor o la representación textual de una constante;
- ★ una referencia a información conservada en otra estructura del compilador;

- ★ datos que permitan relacionar el token con su ubicación en el programa fuente.

No todos los atributos deben estar presentes ni almacenarse de la misma manera. La interfaz se diseña de acuerdo con la información que necesitarán las fases posteriores.

La comunicación entre el analizador sintáctico y el léxico suele organizarse como una solicitud: cuando el analizador sintáctico necesita otra unidad, pide el siguiente token. El analizador léxico consume los caracteres necesarios y devuelve el resultado.



Este modelo evita exigir que toda la secuencia se almacene antes de iniciar el análisis sintáctico. Sin embargo, para estudiar el analizador léxico de manera independiente también es útil recorrer una entrada completa y observar todos los tokens producidos.

Observación 3. Qué hará Lark y qué debemos especificar

En la primera práctica utilizaremos Lark para apoyar la implementación. Lark podrá leer la entrada y producir tokens a partir de las reglas proporcionadas, pero no decide por sí mismo cuáles son las categorías del lenguaje ni qué forma tiene cada una. Esas decisiones deben especificarse y justificarse. Para observar el analizador léxico de manera independiente podrá utilizarse Lark con `parser=None`, `lexer="basic"` y el método `lex()`.

Si ningún patrón permite reconocer el inicio de la entrada restante, el analizador no puede producir el siguiente token. Esa situación corresponde a un problema léxico. Su diagnóstico, recuperación y prueba se desarrollarán en otra nota.

Mini-checkpoint. Del texto al token

Considera el fragmento ilustrativo `total = 25;`. Sin asumir que pertenece a IMP, comprueba si puedes:

1. separar los lexemas `total`, `=`, `25` y `;`;
2. proponer un nombre de token para cada lexema;
3. describir en lenguaje natural el patrón de los identificadores y el de los enteros;
4. señalar cuáles tokens necesitan un atributo para distinguir su aparición concreta.

Conclusión

El analizador léxico establece la primera interfaz del compilador: transforma un flujo de caracteres en una secuencia de tokens que el analizador sintáctico puede procesar. Durante esa transformación agrupa caracteres en lexemas, clasifica cada aparición y conserva los atributos que serán necesarios después. También puede omitir espacios o comentarios cuando las reglas del lenguaje así lo establecen y apoyar la ubicación de los elementos del programa fuente.

Token, lexema y patrón describen aspectos diferentes. El lexema es el texto concreto; el patrón describe la forma admitida; y el token comunica un nombre y, cuando se necesita, un atributo. Esta distinción

permitirá pasar de descripciones informales a especificaciones precisas. En otra nota estudiaremos cómo expresar patrones mediante expresiones regulares y cómo reconocerlos con autómatas finitos.

Referencias

- [1] Aho, A. V., Lam, M. S., Sethi, R., y Ullman, J. D. (2008). *Compiladores: principios, técnicas y herramientas* (2.^a ed.). Pearson Educación. ISBN 978-970-26-1133-2.
- [2] Vilar Torres, J. M. (2010). *Analizador léxico*. Universitat Jaume I.
- [3] Thain, D. (2026). *Introduction to Compilers and Language Design* (2.^a ed.). University of Notre Dame.